

Práctico 3 – Bruno González – Base de Datos 2

Profesor: Cesar Augusto Barrionuevo Saavedra

- 1) Cada vez que se registre una venta, el stock del producto vendido debe reducirse automáticamente en la cantidad vendida.

```
DELIMITER $$
CREATE TRIGGER trg_descontar_stock
AFTER INSERT
ON ventas
FOR EACH ROW
BEGIN
    -- UPDATE productos
    -- SET stock = stock - NEW.cantidad
    -- WHERE id_producto = NEW.id_producto;
END$$
DELIMITER ;
```

- 2) Al insertar una venta, el campo total debe calcularse automáticamente multiplicando cantidad por el precio del producto.

```
DELIMITER $$
CREATE TRIGGER trg_calcular_total
BEFORE INSERT
ON ventas
FOR EACH ROW
BEGIN
    -- DECLARE vprecio DECIMAL(10,2);
    -- SELECT precio INTO vprecio FROM productos WHERE id_producto = NEW.id_producto;
    -- SET NEW.total = NEW.cantidad * vprecio;
END$$
DELIMITER ;
```

- 3) Cuando el precio de un producto cambie, registrar automáticamente en auditoria_precios el precio anterior, el nuevo precio, el usuario MySQL y la fecha.

```
DELIMITER $$
CREATE TRIGGER trg_cambioPrecio
AFTER UPDATE
ON productos
FOR EACH ROW
BEGIN
    -- IF NEW.precio != OLD.precio THEN
        -- INSERT INTO auditoria_precios(precio_anterior, precio_nuevo, usuario, fecha_cambio) VALUES(OLD.precio, NEW.precio, USER());
    -- END IF;
END$$
DELIMITER ;
```

- 4) Antes de eliminar un cliente, guardar sus datos en log_clientes_eliminados para mantener un registro histórico.

```
DELIMITER $$
CREATE TRIGGER trg_guardar_clientes_eliminados
BEFORE DELETE
ON clientes
FOR EACH ROW
BEGIN
    -- INSERT INTO log_clientes_eliminados(id_cliente, nombre, email, fecha_eliminacion) VALUES(OLD.id_cliente, OLD.nombre, OLD.email, NOW());
END$$
DELIMITER ;
```

- 5) Antes de registrar una venta, verificar que exista suficiente stock. Si el stock es insuficiente, abortar la operación con un mensaje de error.

```
DELIMITER $$
CREATE TRIGGER trg_verificar_stock
BEFORE INSERT
ON ventas
FOR EACH ROW
BEGIN
    -- DECLARE vstock INT;
    -- SELECT stock INTO vstock FROM productos WHERE id_producto = NEW.id_producto;
    -- IF cantidad > vstock THEN
        -- SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: El stock no es suficiente.';
    -- END IF;
END $$
DELIMITER ;
```

- 6) Cuando se registra una venta, descontar el total del saldo disponible del cliente automáticamente.

```
DELIMITER $$
CREATE TRIGGER trg_descontar_saldo_cliente
AFTER INSERT
ON ventas
FOR EACH ROW
BEGIN
    -- UPDATE clientes
    -- SET NEW.saldo = saldo - NEW.total
    -- WHERE id_cliente = NEW.id_cliente;
END $$
DELIMITER ;
```

- 7) Antes de actualizar el email de un cliente, normalizar a minúsculas y verificar que no exista otro cliente con ese email.

```
DELIMITER $$
CREATE TRIGGER trg_actualizar_email
BEFORE UPDATE
ON clientes
FOR EACH ROW
BEGIN
    -- SET NEW.email = LOWER(NEW.email);

    -- IF EXISTS (SELECT 1 FROM clientes WHERE email = NEW.email AND id_cliente != OLD.id_cliente) THEN
        -- SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tu email coincide con el de otro cliente.';
    -- END IF;
END $$
DELIMITER ;
```

- 8) Cuando se elimine una venta (cancelación), devolver automáticamente la cantidad al stock del producto correspondiente.

```
DELIMITER $$
CREATE TRIGGER trg_restaurar_stock
AFTER DELETE
ON ventas
FOR EACH ROW
BEGIN
    -- UPDATE productos
    -- SET stock = stock + OLD.cantidad
    -- WHERE id_producto = OLD.id_producto;
END$$
DELIMITER ;
```

- 9) Insertar un nuevo cliente y registrar su primera compra. Si ocurre cualquier error, deshacer todo.

```
START TRANSACTION;
INSERT INTO clientes(nombre, email, saldo) VALUES('Ana Torres', 'ana@email.com', 1000);
SET @id = last_insert_id();
INSERT INTO VENTAS(id_cliente, id_producto, cantidad, total) VALUES(@id, 1, 2, 150.00);
COMMIT;
ROLLBACK;
```

- 10) Implementar una transferencia entre dos cuentas. Validar que el saldo sea suficiente.

```
START TRANSACTION;
-- SET AUTOCOMMIT = 0;
-- UPDATE cuentas SET saldo = saldo - 300 WHERE id_cliente = 1;
-- UPDATE cuentas SET saldo = saldo + 300 WHERE id_cliente = 4;
COMMIT;
ROLLBACK;
```

11) Registrar un pedido con múltiples pasos. Usar SAVEPOINTS para poder deshacer pasos individuales.

```
START TRANSACTION;
-- SET AUTOCOMMIT = 0;

-- SELECT id_cliente INTO @id_cliente FROM clientes WHERE id_cliente = last_insert_id();
-- SAVEPOINT sp_obtener_id_cliente;

-- SELECT total INTO @total FROM ventas WHERE id_cliente = last_insert_id();
-- SAVEPOINT sp_obtener_total;

-- SELECT estado INTO @estado FROM pedidos WHERE id_cliente = last_insert_id();
-- SAVEPOINT sp_obtener_estado;

-- INSERT INTO pedidos(id_cliente, total, estado) VALUES(@id_cliente, @total, @estado);
COMMIT;
ROLLBACK;
```

12) Insertar un lote de 5 productos en una sola transacción usando autocommit deshabilitado.

```
START TRANSACTION;
-- SET AUTOCOMMIT = 0;

INSERT INTO productos(nombre, precio, stock) VALUES ('Remera Supreme', 150000.00, 15),
('Cartuchera Mundial 2022', 340000.00, 25),
('Mouse Gamer RGB', 950000.00, 5),
('Goma de borrar', 1530000.00, 115),
('Galletitas Fauna', 250000.00, 415);

COMMIT;
ROLLBACK;
```

13) TEMA NO VISTO

- 14) Simular un sistema de reserva donde dos usuarios intentan reservar el ultimo asiento disponible simultáneamente.

```
START TRANSACTION;
-- SET AUTOCOMMIT = 0;
-- SELECT disponible FROM asientos WHERE id_asiento = 10 FOR UPDATE;
-- UPDATE asientos SET disponible = 0 WHERE id_asiento = 10;
COMMIT;
ROLLBACK;
```

- 15) TEMA NO VISTO

- 16) Implementar el proceso completo de una compra: verificar stock, registrar venta, actualizar stock y debitar saldo del cliente.

```
START TRANSACTION;
-- SET AUTOCOMMIT = 0;
-- SELECT id_producto INTO @id_producto FROM productos WHERE nombre = 'Remera Supreme';
-- SELECT precio INTO @precio FROM productos WHERE nombre = 'Remera Supreme';
-- SELECT stock INTO @stock FROM productos WHERE nombre = 'Remera Supreme';

-- UPDATE productos
-- SET stock = stock - 10
-- WHERE id_producto = @id_producto AND stock >= 10;

-- SET @total = @precio * 10;

-- INSERT INTO ventas(id_cliente, id_producto, cantidad, total) VALUES(1, @id_producto, 10, @total);

-- UPDATE clientes
-- SET saldo = saldo - @total
-- WHERE id_cliente = 1;
COMMIT;
ROLLBACK;
```